

PANDAS

Pandas is python library, used to analyse data

Example:

Load a CSV file into a Pandas DataFrame:

```
Import pandas as pd
df = pd.read_csv('data.csv')
print(df.to_string())
```

- *pandas* in short called as: **panel data**
- Pandas can be used to remove rows or cells that contains wrong or empty values, this is called: **cleaning**
- True or false. Pandas can be used to find out if there is a correlation between two or more columns: **true**

What is Pandas?

- Pandas is a Python library used for working with data sets.
- It has functions for analyzing, cleaning, exploring, and manipulating data.
- The name "Pandas" has a reference to both "Panel Data", and "Python Data Analysis" and was created by Wes McKinney in 2008.

Why Use Pandas?

- Pandas allows us to analyze big data and make conclusions based on statistical theories.
- Pandas can clean messy data sets, and make them readable and relevant.
- Relevant data is very important in data science.

What Can Pandas Do?

Pandas gives you answers about the data. Like:

- Is there a correlation between two or more columns?
- What is average value?
- Max value?
- Min value?

Pandas are also able to delete rows that are not relevant, or contains wrong values, like empty or NULL values. This is called cleaning the data.

Installation of Pandas : C:\Users\Your Name>pip install pandas

Import Pandas : Once Pandas is installed, import it in your applications by adding the import keyword:

```
Import pandas as pd
mydataset= { 'cars' : ["BMW", "TATA", "AUDI"],
             'passings' : [5, 4, 6]
           }
myvar = pd.DataFrame(mydataset)
print(myvar)
```

Checking Pandas Version : The version string is stored under `__version__` attribute.

```
Import pandas as pd
```

```
Print(pd._version_)
```

- **The official Pandas library is imported like this:** `import pandas`
- **What is a correct way of importing Pandas as an alias?:** `import pandas as pd`

What is a Series? : A Pandas Series is like a column in a table.
It is a one-dimensional array holding data of any type.

```
import pandas as pd
a = [1, 7, 2]
myvar = pd.Series(a)
print(myvar)
```

Labels : If nothing else is specified, the values are labeled with their index number. First value has index 0, second value has index 1 etc.

This label can be used to access a specified value.

`print(myvar[0])` - Return the first value of the Series

```
import pandas as pd
a = [1, 7, 2]
myvar = pd.Series(a)
print(myvar[0]) # o/p - 1
```

Create Labels : With the index argument, you can name your own labels.

```
import pandas as pd
a = [1, 7, 2]
myvar = pd.Series(a, index = ["x", "y", "z"])
print(myvar) #o/p
x    1
y    7
z    2
dtype: int64
```

we can access an item by referring to the label. : `print(myvar["y"])` # 7

Key/Value Objects as Series :

You can also use a key/value object, like a dictionary, when creating a Series.

```
import pandas as pd
calories = {"day1": 420, "day2": 380, "day3": 390}
myvar = pd.Series(calories)
print(myvar)
# o/p :
day1    420
day2    380
day3    390
dtype: int64
```

DataFrames : Data sets in Pandas are usually multi-dimensional tables, called DataFrames.

Series is like a column, a DataFrame is the whole table.
Create a DataFrame from two Series:

```
Import pandas as pd
data = {
    "calories" : [420, 380, 390],
    "duration" : [50, 45, 40]
}
myvar = pd.DataFrame(data)
Print(myvar)
#o/p
   calories  duration
0      420       50
1      380       40
2      390       45
```

- *True or False. A Series is like a row in a table.:* **False**
- *Insert the correct Pandas method to create a Series.: pd.* Series (mylist)
- *Insert the correct syntax to return the first value of a Pandas Series called "myseries"?:* **myseries[0]**
- *Insert the correct syntax to add the labels "x", "y", and "z" to a Pandas Series.***pd.Series(mylist,index=["x","y","z"])**

What is a DataFrame

A Pandas DataFrame is a 2 dimensional data structure, like a 2 dimensional array, or a table with rows and columns.

```
Import pandas as pd
mylist = {
    "calories" : [420,380,300],
    "duration" : [50,40,45]
}
data = pd.DataFrame(mylist)
print(data)
#o/p
   calories  duration
0      420       50
1      380       40
2      390       45
```

Locate Row : As you can see from the result above, the DataFrame is like a table with rows and columns.

Pandas use the loc attribute to return one or more specified row(s)

#refer to the row index:

```
print(df.loc[0])
```

#o/p

calories 420

duration 50

Name: 0, dtype: int64

Note: This example returns a Pandas **Series**.

#use a list of indexes:

```
print(df.loc[[0, 1]])
```

```
calories  duration
0      420      50
1      380      40
```

Note: When using [], the result is a Pandas **DataFrame**.

Named Indexes : With the index argument, you can name your own indexes.

```
Import pandas as pd
```

```
data = {
```

```
    "calories": [450,400,540]
```

```
    "duration": [8,6,5]
```

```
}
```

```
df = pd.DataFrame(data , index = [data, index["da1", "da2","da3"]])
```

```
print(df)
```

```
calories  duration
day1      420      50
day2      380      40
day3      390      45
```

Locate Named Indexes

```
print(df.loc["day2"])
```

```
#o/p
```

```
calories    380
```

```
duration     40
```

```
Name: day2, dtype: int64
```

Load Files Into a DataFrame : If your data sets are stored in a file, Pandas can load them into a DataFrame.

```
Import pandas as pd
```

```
df= pd.read_csv('data.csv')
```

```
print(df)
```

Read CSV Files

A simple way to store big data sets is to use CSV files (comma separated files).

CSV files contains plain text and is a well know format that can be read by everyone including Pandas.

In our examples we will be using a CSV file called 'data.csv'.

```
Import pandas as pd
```

```
df = pd.read_csv('data.csv')
```

```
print(df.to_string())
```

Tip: use to_string() to print the entire DataFrame.

If you have a large DataFrame with many rows, Pandas will only return the first 5 rows, and the last 5 rows:
Print the DataFrame without the `to_string()` method:

```
Import pandas as pd
df = pd.read_csv(data.csv)
print(df)
```

	Duration	Pulse	Maxpulse	Calories
0	60	110	130	409.1
1	60	117	145	479.0
2	60	103	135	340.0
3	45	109	175	282.4
4	45	117	148	406.0
..	...	...	...	...
164	60	105	140	290.8
165	60	110	145	300.4
166	60	115	145	310.2
167	75	120	150	320.4
168	75	125	150	330.4

[169 rows x 4 columns]

max rows : The number of rows returned is defined in Pandas option settings.

You can check your system's maximum rows with the **pd.options.display.max_rows** statement.

Check the number of maximum returned rows:

```
import pandas as pd
print(pd.options.display.max_rows)
```

#o/p

```
PS C:\Users\MEGHA\Desktop\LLM-Project> & "C:\Program Files\Python312\python.exe"
```

```
c:/Users/MEGHA/Desktop/LLM-Project/max_rows.py
```

60

In my system the number is 60, which means that if the DataFrame contains more than 60 rows, the print(df) statement will return only the headers and the first and last 5 rows.

You can change the maximum rows number with the same statement.

```
Import pandas as pd
Pd.options.display.max_rows = 9999
df = pd.read_csv('data.csv')
print(df)
```

- CSV is short for: **comma separated values**
- What is a correct syntax for loading a .csv file into a DataFrame?: **df=pd.read_csv('data.csv')**
- By default, if you print a large DataFrame with the `print(df)` function, Pandas will only return a selection of the data. How many rows will be returned?: **10, the headers and the first and last 5 rows**

Read JSON

Big data sets are often stored, or extracted as JSON.

JSON is plain text, but has the format of an object, and is well known in the world of programming, including Pandas.

Load the JSON file into a DataFrame:

```
import pandas as pd
df = pd.read_json('data.json')
print(df.to_string())
```

Tip: use `to_string()` to print the entire DataFrame.

Dictionary as JSON

JSON = Python Dictionary

JSON objects have the same format as Python dictionaries.

If your JSON code is not in a file, but in a Python Dictionary, you can load it into a DataFrame directly:

```
import pandas as pd

data = {
    "Duration":{
        "0":60,
        "1":60,
        "2":60,
        "3":45,
        "4":45,
        "5":60
    },
    "Pulse":{
        "0":110,
        "1":117,
        "2":103,
        "3":109,
        "4":117,
        "5":102
    },
    "Maxpulse":{
        "0":130,
        "1":145,
        "2":135,
        "3":175,
        "4":148,
        "5":127
    },
    "Calories":{
        "0":409,
        "1":479,
        "2":340,
        "3":282,
        "4":406,
        "5":300
    }
}

df = pd.DataFrame(data)
print(df)
```

#o/p

	Duration	Pulse	Maxpulse	Calories
0	60	110	130	409.1
1	60	117	145	479.0
2	60	103	135	340.0
3	45	109	175	282.4
4	45	117	148	406.0
5	60	102	127	300.5

- What is a correct syntax for loading a .json file into a DataFrame?: **`df = pd.read_json('data.json')`**
- JSON objects have the same format as a certain Python object, which one?:

Python Dictionaries

- True or False. Python Dictionaries can be loaded directly into a Pandas DataFrame.: ***true***

Pandas - Analyzing DataFrames

Viewing the Data

- One of the most used method for getting a quick overview of the DataFrame, is the head() method.
- The head() method returns the headers and a specified number of rows, starting from the top.

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df.head(10))
```

	Duration	Pulse	Maxpulse	Calories					
					5	60	102	127	300.5
0	60	110	130	409.1	6	60	110	136	374.0
1	60	117	145	479.0	7	45	104	134	253.3
2	60	103	135	340.0	8	30	109	133	195.1
3	45	109	175	282.4	9	60	98	124	269.0
4	45	117	148	406.0					

Note: if the number of rows is not specified, the head() method will return the top 5 rows.

Print the first 5 rows of the DataFrame:

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df.head())
```

- There is also a tail() method for viewing the *last* rows of the DataFrame.
- The tail() method returns the headers and a specified number of rows, starting from the bottom.

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df.tail())
```

	Duration	Pulse	Maxpulse	Calories
164	60	105	140	290.8
165	60	110	145	300.4
166	60	115	145	310.2
167	75	120	150	320.4
168	75	125	150	330.4

Info About the Data:

The DataFrames object has a method called info(), that gives you more information about the data set.

Print information about the data:

```
Import pandas as pd
df = pd.read_csv('data.csv')
print(df.info())
```

#o/p

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 169 entries, 0 to 168

Data columns (total 4 columns):

#	Column	Non-Null Count	Dtype
0	Duration	169 non-null	int64
1	Pulse	169 non-null	int64
2	Maxpulse	169 non-null	int64
3	Calories	164 non-null	float64

dtypes: float64(1), int64(3)

memory usage: 5.4 KB

None

Result Explained

The result tells us there are 169 rows and 4 columns:

RangeIndex: 169 entries, 0 to 168

Data columns (total 4 columns):

And the name of each column, with the data type:

#	Column	Non-Null Count	Dtype
0	Duration	169 non-null	int64
1	Pulse	169 non-null	int64
2	Maxpulse	169 non-null	int64
3	Calories	164 non-null	float64

Null Values

- The info() method also tells us how many Non-Null values there are present in each column, and in our data set it seems like there are 164 of 169 Non-Null values in the "Calories" column.
- Which means that there are 5 rows with no value at all, in the "Calories" column, for whatever reason.
- Empty values, or Null values, can be bad when analyzing data, and you should consider removing rows with empty values. This is a step towards what is called *cleaning data*, and you will learn more about that in the next chapters.
- What is a correct syntax for printing the first 10 rows of a DataFrame? : ***print(df.head(10))***
- What is a correct syntax for printing the last 10 rows of a DataFrame? : ***print(df.tail(10))***
- True or False. If number of rows is not specified, the tail() method will return all rows. : ***false***

Data Cleaning:

Data cleaning means fixing bad data in your data set.

Bad data could be:

- Empty cells
- Data in wrong format
- Wrong data
- Duplicates

Our Data Set

Duration	Date	Pulse	Maxpulse	Calories	0	60	'2020/12/01'	110	130	409.1
----------	------	-------	----------	----------	---	----	--------------	-----	-----	-------

1	60	'2020/12/02'	117	145	479.0	17	60	'2020/12/17'	100	120	300.0
2	60	'2020/12/03'	103	135	340.0	18	45	'2020/12/18'	90	112	NaN
3	45	'2020/12/04'	109	175	282.4	19	60	'2020/12/19'	103	123	323.0
4	45	'2020/12/05'	117	148	406.0	20	45	'2020/12/20'	97	125	243.0
5	60	'2020/12/06'	102	127	300.0	21	60	'2020/12/21'	108	131	364.2
6	60	'2020/12/07'	110	136	374.0	22	45	NaN	100	119	282.0
7	450	'2020/12/08'	104	134	253.3	23	60	'2020/12/23'	130	101	300.0
8	30	'2020/12/09'	109	133	195.1	24	45	'2020/12/24'	105	132	246.0
9	60	'2020/12/10'	98	124	269.0	25	60	'2020/12/25'	102	126	334.5
10	60	'2020/12/11'	103	147	329.3	26	60	2020/12/26	100	120	250.0
11	60	'2020/12/12'	100	120	250.7	27	60	'2020/12/27'	92	118	241.0
12	60	'2020/12/12'	100	120	250.7	28	60	'2020/12/28'	103	132	NaN
13	60	'2020/12/13'	106	128	345.3	29	60	'2020/12/29'	100	132	280.0
14	60	'2020/12/14'	104	132	379.3	30	60	'2020/12/30'	102	129	380.3
15	60	'2020/12/15'	98	123	275.0	31	60	'2020/12/31'	92	115	243.0
16	60	'2020/12/16'	98	120	215.2						

The data set contains some empty cells ("Date" in row 22, and "Calories" in row 18 and 28).

The data set contains wrong format ("Date" in row 26).

The data set contains wrong data ("Duration" in row 7).

The data set contains duplicates (row 11 and 12).

EMPTY CELLS: Empty cells can potentially give you a wrong result when you analyze data.

- **Remove Rows**

One way to deal with empty cells is to remove rows that contain empty cells.

This is usually OK, since data sets can be very big, and removing a few rows will not have a big impact on the result.

```
import pandas as pd
df = pd.read_csv('data.csv')
new_df = df.dropna()
print(new_df.to_string())
```

Duration	Date	Pulse	Maxpulse	Calories	12	60	'2020/12/12'	100	120	250.7	
0	60	'2020/12/01'	110	130	409.1	13	60	'2020/12/13'	106	128	345.3
1	60	'2020/12/02'	117	145	479.0	14	60	'2020/12/14'	104	132	379.3
2	60	'2020/12/03'	103	135	340.0	15	60	'2020/12/15'	98	123	275.0
3	45	'2020/12/04'	109	175	282.4	16	60	'2020/12/16'	98	120	215.2
4	45	'2020/12/05'	117	148	406.0	17	60	'2020/12/17'	100	120	300.0
5	60	'2020/12/06'	102	127	300.0	19	60	'2020/12/19'	103	123	323.0
6	60	'2020/12/07'	110	136	374.0	20	45	'2020/12/20'	97	125	243.0
7	450	'2020/12/08'	104	134	253.3	21	60	'2020/12/21'	108	131	364.2
8	30	'2020/12/09'	109	133	195.1	23	60	'2020/12/23'	130	101	300.0
9	60	'2020/12/10'	98	124	269.0	24	45	'2020/12/24'	105	132	246.0
10	60	'2020/12/11'	103	147	329.3	25	60	'2020/12/25'	102	126	334.5
11	60	'2020/12/12'	100	120	250.7	26	60	2020/12/26	100	120	250.0

27	60	'2020/12/27'	92	118	241.0	30	60	'2020/12/30'	102	129	380.3
29	60	'2020/12/29'	100	132	280.0	31	60	'2020/12/31'	92	115	243.0

#Notice in the result that some rows have been removed (row 18, 22 and 28).

#These rows had cells with empty values.

Note: By default, the dropna() method returns a new DataFrame, and will not change the original.

- If you want to change the original DataFrame, use the inplace = True argument:

Remove all rows with NULL values:

```
import pandas as pd
df = pd.read_csv('data.csv')
df.dropna(inplace = True)
print(df.to_string())
```

Note: Now, the dropna(inplace = True) will NOT return a new DataFrame, but it will remove all rows containing NULL values from the original DataFrame.

Replace Empty Values

- Another way of dealing with empty cells is to insert a *new* value instead.
- This way you do not have to delete entire rows just because of some empty cells.
- The fillna() method allows us to replace empty cells with a value:

```
import pandas as pd
df = pd.read_csv('data.csv')
df.fillna(130, inplace = True)
print(df.to_string())
```

#Notice in the result: empty cells got the value 130 (in row 18, 22 and 28).

18	45	'2020/12/18'	90	112	130.0
22	45	130	100	119	282.0
28	60	'2020/12/28'	103	132	130.0

- **Replace Only For Specified Columns**

The example above replaces all empty cells in the whole Data Frame.

To only replace empty values for one column, specify the *column name* for the DataFrame:

Replace NULL values in the "Calories" columns with the number 130:

```
import pandas as pd
df = pd.read_csv('data.csv')
df.fillna({"Calories": 130}, inplace=True)
```

18	45	'2020/12/18'	90	112	130.0
28	60	'2020/12/28'	103	132	130.0

- **Replace Using Mean, Median, or Mode**

A common way to replace empty cells, is to calculate the mean, median or mode value of the column.

Pandas uses the mean() median() and mode() methods to calculate the respective values for a specified column:

- **Calculate the MEAN, and replace any empty values with it:**

```
import pandas as pd
df = pd.read_csv('data.csv')
x = df["Calories"].mean()
df.fillna({"Calories": x}, inplace=True)
```

#As you can see in row 18 and 28, the empty values from "Calories" was replaced with the mean: 304.68

18	45	'2020/12/18'	90	112	304.68
----	----	--------------	----	-----	--------

8	60	'2020/12/28'	103	132	304.68
---	----	--------------	-----	-----	--------

Mean = the average value (the sum of all values divided by number of values).

- Calculate the **MEDIAN**, and replace any empty values with it:

```
import pandas as pd
df = pd.read_csv('data.csv')
x = df["Calories"].median()
df.fillna({"Calories": x}, inplace=True)
```

#As you can see in row 18 and 28, the empty values from "Calories" was replaced with the median: 291.2

18	45	'2020/12/18'	90	112	291.2
----	----	--------------	----	-----	-------

28	60	'2020/12/28'	103	132	291.2
----	----	--------------	-----	-----	-------

Median = the value in the middle, after you have sorted all values ascending

- Calculate the **MODE**, and replace any empty values with it:

```
import pandas as pd
df = pd.read_csv('data.csv')
x = df["Calories"].mode()[0]
df.fillna({"Calories": x}, inplace=True)
```

#As you can see in row 18 and 28, the empty value from "Calories" was replaced with the mode: 300.0

18	45	'2020/12/18'	90	112	300.0
----	----	--------------	----	-----	-------

28	60	'2020/12/28'	103	132	300.0
----	----	--------------	-----	-----	-------

Mode = the value that appears most frequently.

- What is the correct explanation of the MEDIAN value?: The value in the middle, after you have sorted all values ascending.
- Insert the correct syntax for removing rows with empty cells.: `df.dropna()`
- Insert the correct syntax for replacing empty cells with the value "130".: `df.fillna(130)`
- Insert the correct argument to make sure that the changes are done for the original DataFrame instead of returning a new one.: `dropna(inplace = True)`

Cleaning Data of Wrong Format

- Cells with data of wrong format can make it difficult, or even impossible, to analyze data.
- To fix it, you have two options: remove the rows, or convert all cells in the columns into the same format.

Convert Into a Correct Format

In our Data Frame, we have two cells with the wrong format. Check out row 22 and 26, the 'Date' column should be a string that represents a date:

22	45	NaN	100	119	282.0
----	----	-----	-----	-----	-------

```
26    60    20201226    100    120    250.0
```

- Pandas has a `to_datetime()` method for this:

```
import pandas as pd
df = pd.read_csv('data.csv')
df['Date'] = pd.to_datetime(df['Date'], format='mixed')
print(df.to_string())
```

```
22    45      NaT    100    119    282.0
26    60 '2020/12/26'    100    120    250.0
```

As you can see from the result, the date in row 26 was fixed, but the empty date in row 22 got a NaT (Not a Time) value, in other words an empty value. One way to deal with empty values is simply removing the entire row.

- **Removing Rows**

```
import pandas as pd
df = pd.read_csv('data.csv')
df['Date'] = pd.to_datetime(df['Date'], format='mixed')
df.dropna(subset=['Date'], inplace = True)
print(df.to_string())
```

```
26    60 2020-12-26    100    120    250.0
```

- What is a correct method for converting a column into date formats?: `to_datetime ()`
- True or False. Cells with different formats can be difficult to compare.: true

Fixing Wrong Data

- "Wrong data" does not have to be "empty cells" or "wrong format", it can just be wrong, like if someone registered "199" instead of "1.99".
- If you take a look at our data set, you can see that in row 7, the duration is 450, but for all the other rows the duration is between 30 and 60.
- It doesn't have to be wrong, but taking in consideration that this is the data set of someone's workout sessions, we conclude with the fact that this person did not work out in 450 minutes.

```
7    450 '2020/12/08'    104    134    253.3
```

- **Replacing Values**

one way to fix wrong values is to replace them with something else.

In our example, it is most likely a typo, and the value should be "45" instead of "450", and we could just insert "45" in row 7:

```
import pandas as pd
df = pd.read_csv('data.csv')
df.loc[7, 'Duration'] = 45
print(df.to_string())
```

18 45 '2020/12/18' 90 112 NaN

- To replace wrong data for larger data sets you can create some rules, e.g. set some boundaries for legal values, and replace any values that are outside of the boundaries.

Loop through all values in the "Duration" column.

If the value is higher than 120, set it to 120:

```
import pandas as pd
df = pd.read_csv('data.csv')
for x in df.index:
    if df.loc[x, "Duration"] > 120:
        df.loc[x, "Duration"] = 120
print(df.to_string())
```

Removing Rows

- Another way of handling wrong data is to remove the rows that contains wrong data.
- This way you do not have to find out what to replace them with, and there is a good chance you do not need them to do your analyses.

Delete rows where "Duration" is higher than 120:

```
import pandas as pd
df = pd.read_csv('data.csv')
for x in df.index:
    if df.loc[x, "Duration"] > 120:
        df.drop(x, inplace = True)
```

#remember to include the 'inplace = True' argument to make the changes in the original DataFrame object instead of returning a copy

```
print(df.to_string())
```

Removing Duplicates

Duplicate rows are rows that have been registered more than one time.

11 60 '2020/12/12' 100 120 250.7
12 60 '2020/12/12' 100 120 250.7

- To discover duplicates, we can use the duplicated() method.
- The duplicated() method returns a Boolean values for each row:

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df.duplicated())
```

0	False	5	False	10	False
1	False	6	False	11	False
2	False	7	False	12	True
3	False	8	False	13	False
4	False	9	False	14	False

15 False
16 False
17 False
18 False
19 False
20 False

21 False
22 False
23 False
24 False
25 False
26 False

27 False
28 False
29 False
30 False
31 False
dtype: bool

- **Removing Duplicates**

To remove duplicates, use the `drop_duplicates()` method.

```
import pandas as pd
df = pd.read_csv('data.csv')
df.drop_duplicates(inplace = True)
print(df.to_string())
```

#Notice that row 12 has been removed from the result

Remember: The (`inplace = True`) will make sure that the method does NOT return a *new* DataFrame, but it will remove all duplicates from the *original* DataFrame.

Data Correlations

Finding Relationships

- A great aspect of the Pandas module is the `corr()` method.
- The `corr()` method calculates the relationship between each column in your data set.

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df.corr())
```

	Duration	Pulse	Maxpulse	Calories
Duration	1.000000	-0.155408	0.009403	0.922721
Pulse	-0.155408	1.000000	0.786535	0.025120
Maxpulse	0.009403	0.786535	1.000000	0.203814
Calories	0.922721	0.025120	0.203814	1.000000

Note: The `corr()` method ignores "not numeric" columns.

Result Explained

- The Result of the `corr()` method is a table with a lot of numbers that represents how well the relationship is between two columns.

The number varies from -1 to 1.

- 1 means that there is a 1 to 1 relationship (a perfect correlation), and for this data set, each time a value went up in the first column, the other one went up as well.
- 0.9 is also a good relationship, and if you increase one value, the other will probably increase as well.
- -0.9 would be just as good relationship as 0.9, but if you increase one value, the other will probably go down.
- 0.2 means NOT a good relationship, meaning that if one value goes up does not mean that the other will.

What is a good correlation? It depends on the use, but I think it is safe to say you have to have at least 0.6 (or -0.6) to call it a good correlation.

- **Perfect Correlation:**

We can see that "Duration" and "Duration" got the number 1.000000, which makes sense, each column always has a perfect relationship with itself.

- **Good Correlation:**

"Duration" and "Calories" got a 0.922721 correlation, which is a very good correlation, and we can predict that the longer you work out, the more calories you burn, and the other way around: if you burned a lot of calories, you probably had a long work out.

- **Bad Correlation:**

"Duration" and "Maxpulse" got a 0.009403 correlation, which is a very bad correlation, meaning that we can not predict the max pulse by just looking at the duration of the work out, and vice versa.

Plotting

- Pandas uses the plot() method to create diagrams.
- We can use Pyplot, a submodule of the Matplotlib library to visualize the diagram on the screen.

#Three lines to make our compiler able to draw:

```
import sys
import matplotlib
matplotlib.use('Agg')
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv('data.csv')
df.plot()
plt.show()
```

#Two lines to make our compiler able to draw:

```
plt.savefig(sys.stdout.buffer)
sys.stdout.flush()
```

